Arrays, 2D arrays and more of Nested loops

Sudeep K S, Mahindra University (October 2026)

- **Array**

An 'Array' in C is a collection of variables **of the same type**.

For example, a list of 100 integers (say, marks of 100 students in a class). The memory for all these 100 integer variables get allocated as a single block (space for 100 integer variables).

Each individual item can be referred to using its 'index' (position) in this block.

5	12	10	34	40	6	8
Index = 0	1	2	3	4	5	6

The figure above shows an array of seven integers. Indexing (positions) start from 0.

The syntax of declaring an array is as follows:

```
<type of each item> <array name>[number of elements];
```

For example, an integer array named 'marks' that stores 10 integer values is declared as

```
int marks[10];
```

Such an array can be 'filled' with values using an assignment statement directly as follows.

```
int marks[10] = {18, 5, 10, 15, 6, 0.5, 25, 13, 3, 10};
```

Note that this is similar to assigning a value to a single integer variable. The only difference here is that we are now assigning values to ten variables at a time.

Note that the individual variables can be accessed as `marks[0]`, `marks[1]`, …, up to `marks[9]`.

Each element in array can be treated as a single variable. For example, if we need to add 5 to the fourth number in this array, we can do it as

```
marks[3] = marks[3] + 5;
```

Note that index 3 refers to the 4th element in this array.

Alternatively, we can also read in these values as input from the user, using a sequence of `scanf` statements. It can be done easily with a for loop, as follows.

```
printf("enter the 10 marks (into the marks array) one by one\n");
for (i = 0; i < 10; i++)
     scanf("%d", &marks[i]);
```

The values of the variables can be displayed (printed) using another for loop, as follows:

```
int i;
for (i = 0; i < 10; i++)
     printf("marks of student %d is %d\n", i+1, marks[i]);
```

Similarly, if we want to find the highest element in the array, we can do something like this:

```
int i, max;

max = marks[0]; // initialize 'max' as the first element
for (i = 1; i < 10; i++)
{
   if (marks[i] > max) { // see if the current element is bigger
       max = marks[i]; // if so, update the 'max' (highest element)
   }
}
```

Note that the parentheses (brackets) here are not really needed, as these are single statements.

The **curly parentheses** { } are used to group a sequence of statements and make them look like a single statement. It can be used in `if`, `while`, `for` and `do while`. If grouped like that, all the statements within that block will get executed one after the other, in sequence.

- 2-D (two dimensional) arrays

Just like normal arrays (as seen above), we can also have 2-D arrays (variables that are arranged in rows and columns, like a matrix) or even bigger dimensions. Example:

```
int matrix[10][5]; // 10 x 5 matrix of integers
```

We can declare and initialize the values in a two-dimensional array like this:

```
// Declare and initialize a 2D array with 3 rows and 3 columns
int matrix[3][3] = {
        {1, 2, 3},  // Row 1: elements matrix[0][0], matrix[0][1], matrix[0][2]
        {4, 5, 6},  // Row 2: elements matrix[1][0], matrix[1][1], matrix[1][2]
        {7, 8, 9}   // Row 3: elements matrix[2][0], matrix[2][1], matrix[2][2]
};
```

Alternatively, we can ask the user to enter the values and read them into the memory. See below program that reads two 3x3 matrices and computes their sum. It uses nested for loops.

```
#include <stdio.h>
int main() {

    int i, j;

    //declare three 3x3 matrices, two for input, one for result
    int matrixA[3][3];
    int matrixB[3][3];
    int matrixC[3][3];

    // ask the user to enter the values in matrix A, and use scanf to read values
    printf("enter the elements of matrix A one by one\n");
    for (i = 0; i < 3; i++)
       for (j = 0; j < 3; j++)
          scanf("%d", &matrixA[i][j]);

    // ask the user to enter the values in matrix B, and read
    printf("enter the elements of matrix B one by one\n");
    for (i = 0; i < 3; i++)
       for (j = 0; j < 3; j++)
          scanf("%d", &matrixB[i][j]);

    // find sum: C = A + B
    for (i = 0; i < 3; i++)
       for (j = 0; j < 3; j++)
          matrixC[i][j] = matrixA[i][j] + matrixB[i][j];

    // print the result matrix, C
    printf("matrix C as sum of A and B is\n");
    for (i = 0; i < 3; i++)
    {
       for (j = 0; j < 3; j++)
          printf("%d ", matrixC[i][j]);
       printf("\n"); // for breaking the line after each row
    }

    return 0;
}
```

Note that technically the way the computer allocates space for an 10 x 5 array is exactly the same as a one-dimensional array of size 50: that is, enough space for 50 variables, one after the other.

However, it adds convenience to those who write (and read) the program, because a lot of data can be visualized as rows and columns rather than just a sequence of many numbers, and we can access them by giving the row and column index values, as m[i][j], for computations.